Assignment 3

1. 📝 **Worth:** 6%
2. 📅 **Due Date:** See LEA
3. 🕑 **Late submissions:** 10% penalty per late day.
4. 📤 **Submission:**
    - Put all `.py` file(s) into a `.zip` and submit on LEA
    - Please use the provided files **WITHOUT** renaming them.
    - Fill the docstring header at the top of each `.py` file with your name, section and date.
    - Use type hinting on all functions.
    - Use the `my_functions.py` to define all your functions
    - Use the `assignment3.py` file to test all your functions.

Instruction updates

⚠️ Nov 06: To clear up some confusion with the instructions, I've made some updates. Look for the warning symbol (⚠️).

🎯 Learning Objectives

- Break down a problem into smaller solvable steps
- Design and implement an algorithm using the concepts covered in class.
- Use `for` loops for repetition
- Use `if`, `if-else` and `elif` statements for conditional logic
- Implement and use **functions** to improve code structure and reuse logic.

Getting started

⚠️ Nov 6 update: I've added this section to help clarify what code goes in what file.

On LEA I have provided two starter files in a `.zip`:

- `assignment3.py`

- `my_functions.py`

Download the `.zip`, unzip it, and put these two files in a Pycharm project for assignment 3.

In this assignment, you will be required to *define* your functions in `my_functions.py`, and *use* them in `assignment3.py`.

That is, your files should look something like this:

```
# my_functions.py
# ... docstrings omitted ...

def net_income(hourly_rate, hours_per_week):
    # NOTE: this is not the actual solution
    return hourly_rate * hours_per_week


def real_net_income(hourly_rate, hours_per_week):
    # NOTE: this is not the actual solution
    return hourly_rate * hours_per_week



# assignment3.py
# ... docstrings omitted ...

from my_functions import *

def main():
    """Question 1"""
    print(net_income(hourly_rate=14.50, hours_per_week=15))
    print(actual_net_income(hourly_rate=14.50, hours_per_week=15))

    """Question 2"""
    print(intake_frequency(5))

# etc., do the same for the rest of the questions
```

To test that your functions work, *run* the `assignment3.py` code.

> **IMPORTANT:** You should not use any `print()` statements in `my_functions.py` — you should use `return` statements instead.

🏛 Question 1 : Taxes

Write a function to help a person estimate their net income.

Function: `net_income`

- **Input parameter**: `hourly_rate` and `hours_per_week`
- **Returns**: the `net_income` which is calculated as `gross_income * (1-tax_rate)`

> ⚠️ NOTE: Why is `net_income = gross_income * (1-tax_rate)` ? It's easier to understand if you expand the expression:
>
> ```
> gross_income*(1-tax_rate) == gross_income - gross_income*tax_rate
> ```
>
> That is, your `net_income` is whatever is left after you subtract `gross_income*tax_rate` from `gross_income` .

Important Notes

- Convert the **weekly** salary to a **yearly** salary.
- Assume that there is **52 paid** weeks in a year.
- `hourly_rate` and the `hours_per_week` can have decimal values.
- Use `if/elif/else` statements to determine the appropriate tax rate.
- Use the Quebec income tax rates table below to find the `tax_rate` associated with the income range:

⚠️ Note that the table has been updated for 2025 values.

Income range	Tax rate
$53,255 or less	14%
More than $53,255 but not more than $106,495	19%
More than $106,495 but not more than $129,590	24%
More than $129,590	25.75%

Expected output for `net_income`

In your main script, test your function with different `hourly_pay` and `weekly_hours` .

For example, if in `assignment3.py` you have:

```
print(f"Your estimated net income is: ${net_income(14.5, 15.0):.2f}")
print(f"Your estimated net income is: ${net_income(100, 40):.2f}")
```

The expected output should be:

```
Your estimated net income is: $9726.60
Your estimated net income is: $154440.00
```

Make sure you test more cases than just these two!

⚠️ (BONUS) Function: `actual_net_income`

- **Input parameter**: `hourly_rate` and `hours_per_week`
- **Returns**: the actual `net_income` which is calculated as a function of `gross_income` and all applicable `tax_rate` values.

This question is a bonus: it is not required to get 100%, but can boost your mark if you complete it correctly.

Note that the previous question doesn't actually calculate taxes correctly!

In real life, **tax rate** should only be applied *for the portion of income that that rate applies to*, not the entire income.

That is, if someone earns $60,000:

- the first $53,255 is deducted at a rate of 14%
- the following $6,745 is deducted at a rate of 19%

This means that they are taxed less overall (since the 19% rate isn't applied to their entire income, only the portion that falls into that tax bracket). This also makes more sense: someone making $54k shouldn't have a lower gross income than someone making $53k, just because of where the tax bracket line is defined.

Create another function called `actual_net_income` that applies these tax rate corrections correctly.

> HINT: If you use a chain of `if` statements, instead of `if/elif` statements, you can apply a sequence of calculations using each tax rate rather than using just one tax rate.

Expected output for `actual_net_income`

```
print(f"Your estimated net income is: ${net_income(14.5, 15.0):.2f}")
print(f"Your estimated net income is: ${net_income(100, 40):.2f}")
```

The expected output should be:

```
Your estimated net income is: $9726.60
Your estimated net income is: $164695.33
```

Note that for larger incomes, the gross income is larger (since the larger tax rates won't be applied to their entire income).

💊Question 2: Active Substance

The half-life of a substance is the time it takes for its concentration to be reduced to half. In medicine, this determines how often a patient should take a drug. Physicians decide on a dosage schedule by multiplying the half-life by 4 or by 5 which gives a **rough** estimation of the next time the patient should take the medication.

The concentration of a drug in the bloodstream is described by the following formula:

$$C(t) = C_0 e^{-\lambda t}$$

Here λ (`lambda`) is the decay constant specific to each drug, is calculated using this formula:

$$\lambda = \frac{ln(2)}{t_{1/2}}$$

> For example the drug Donepezil used to treat dementia, has a half-life of 70 hours (4,200 minutes). According to the 4-5 * half-life rule used by doctors, this means the drug should be taken every 11-14 days.

The objective is to use `python` to write a script which will determine the intake frequency of a medication with at an accuracy +/- 1 hour.

Function: `concentration_percent`

- **Input parameters:** the `time` in **hours** and the `half_life` of the substance **in hours**
- **returns** the `concentration` in percentage using this formula:

$$\frac{C(t)}{C_0} = e^{-\lambda t}$$

where λ is given by this formula:

$$\lambda = \frac{ln(2)}{t_{1/2}}$$

> ⚠️ **Notes:**
>
> - the time elapsed t and the half-life time $t_{1/2}$ aren't the same!
>   - $t_{1/2}$ i.e. half life: the amount of time it takes for a substance to decay to half its original concentration in general
>   - t i.e. time elapsed: how much time has passed.
> - The point of this function is to ask, given a known a known half-life $t_{1/2}$, what is the concentration of the substance that remains after time t has elapsed?
>   - both t and $t_{1/2}$ can be fractional values.
> - Use the `math` library for the mathematical functions:
>   - `math.log` gives the natural logarithm (`ln()` , i.e. base e) by default
>   - `math.e` gives the Euler's constant e

Expected output for `concentration_percent`

One example: if the half life and the time elapsed are the same, you should get that 50% (0.50) concentration remains:

```
print(f"Concentration percent: ${concentration_percent(1, 1):.2f}")
```

Your result would be:

```
Concentration percent: 0.50
```

Try more numbers for time elapsed/half-life to make sure it works!

Function: `intake_frequency`

Determines how often a drug should be taken based on when its concentration falls **below 5%**.

- **Input parameter(s)**: the `half_life` of a substance (in hours)

- **returns:** the time interval between each dose.

Algorithm

- set `lower_bound` $\leftarrow floor(4t_{1/2})$
- set `upper_bound` $\leftarrow ceil(5t_{1/2})$
- Initialize `intake_hour` $\leftarrow$ `upper_bound`
- If the `upper_bound` _ ≤ 1, _return the `intake_hour`
- For every hour t between `lower_bound` to `upper_bound`
  - Compute $\frac{C(t)}{C_0}$
  - If $\frac{C(t)}{C_0} \leq 5\%$
    - set the `intake_hour` $\leftarrow t$
    - **break**
- return the `intake_hour`

> **Note**:
>
> - The `range()` only takes input values of type `int` . Round the start and end values of the interval using the appropriate rounding functions.
> - Each iteration, use `concentration_percent` to get the remaining concentration in percentage relative to the initial concentration C_0.

Expected output for `intake_frequency`

You can also compare your function's output with the analytical solution for various half times of various substances.

$t_{end} = \frac{ln(20)}{ln(2)} t_{1/2}$

Substance	Half life	Intake frequency: $\frac{ln(20)}{ln(2)} t_{1/2}$
Caffeine	~5 hours	~21.61 hours
Theobromine (found in chocolate)	~7 hours	~30.25 hours
Ibuprofen (Advil)	~1.8 hours	~7.78 hours
Acetaminophen (Tylenol)	2 hours	~8.64 hours

Your function should give similar (not exactly the same) results as the third column.

➗ Question 3: Greatest Common Divisor

In mathematics, the **greatest common divisor** (**GCD**), also known as **greatest common factor (GCF)**, of two or more integers, which are not all zero, is the largest positive integer that divides each of the integers.

For example, the GCD of 8 and 12 is 4, that is, $gcd(8, 12) = 4$

wikipedia

Function: `gcd`

- **Input parameters :**
  - `a` (int)
  - `b` (int)
- **Returns:** the greatest common divisor

Algorithm

Write code that calculates the greatest common divisor using the following algorithm:

Repeat **enough** times:

- *if* $a \leq b$ *, then* $b \leftarrow b - a$
- *if* $b < a$*, then* $a \leftarrow a - b$
- _if a is equal to zero:
  - **return** b
- _if b is equal to zero:
  - **return** a

The restult of `gcd` will be either a or b, whichever is not zero

Expected output for `gcd(a, b)`

In the main script, call the function with various values, for example:

```
print(gcd(105,252))
```

Expected Output:

```
21
```

Try more numbers than this!

🧵 Question 4: Hexagon t-shirts

A company makes fabrics based on the trendy hexagon pattern shown below:

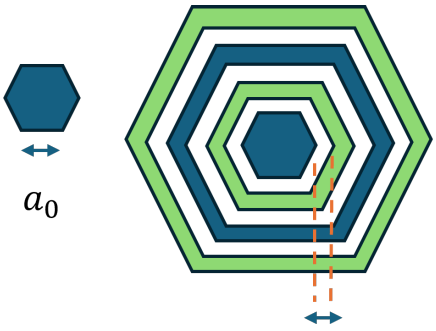
They are currently offering a wide selection of sizes characterized by the number of blue, white and green hexagons (`num_hex`) as shown in the figures below.

num_hex = 6

num_hex = 7

num_hex = 8

All patterns have a central hexagon of side length a_0. The concentric hexagons are evenly spaced by a value of w.



The company has tasked you with creating a script to calculate the area covered by the green fabric given the number of hexagons, the central hexagon's side length and the spacing between consecutive hexagons.

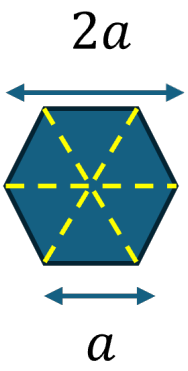
Function: `hex_area`

- **Input parameter :**
  - `side` : Length of a side of the hexagon

- **Returns:**
  - Hexagon's area using the formula:

$$\frac{3\sqrt{(3)}}{2}a^2$$

Assume that all hexagons are regular and where a is the side of the hexagon as shown in the figure below:



Expected output for `hex_area`

In your `assignment3.py`, try out different size hexagons:

```
print(f"Area for hex side length 1: {hex_area(1):.2f}")
print(f"Area for hex side length 10: {hex_area(10):.2f}")
```

For the examples above, you should get the following:

```
Area for hex side length 1: 2.60
Area for hex side length 10: 259.81
```

Make sure your hex function works on any size hexagon (try out a few more numbers and make sure the result makes sense).

Function: `green_area`

- **Input parameter:**
  - `num_hex` : total number of hexagons of the printed pattern
  - `a0` : the length of side of the central hexagon
  - `w` : the spacing between consecutive hexagons w
- **Returns:**
  - Area covered by the green fabric.

> Hint: Start by determining the relationship between two consecutive hexagons' side length.

Expected output for `green_area`

In the main script, test your function with various values:

```
print("Area for 6 hexagons: ", green_area(6,3.0,1.0))
print("Area for 7 hexagons: ", green_area(7,3.0,1.0))
print("Area for 8 hexagons: ", green_area(8,3.0,1.0))
```

Try different size central hexagons/spacing values as well and make sure the results make sense.

Grading Ruberic

Criteria	Description	Points
Name and Id	Docstring file descriptions are completed on each file	1
Code Quality	Use of variables and constants to avoid magical numbers. Naming conventions are respected. Import statements are defined at the top of each file. Code is easy to read and equations are broken down when necessary. All functions are defined at the same indentation level.	5
Conditional statements	Correct and optimal use of comparison operators and conditional statements. Avoids redundant conditions.	10
Functions	All functions are defined in the `my_functions.py` script. Correct calculations are performed	10
Loops	Correct use of loops and `break` or `return` statement when necessary	15

Criteria	Description	Points
Logic & algorithm	Correct translation of a problem into logical and broken down steps to achieve the goal	20
Comments	Uses comments to explain logic when necessary.	5